

Final Project – Deep Learning - 99006

General Instructions

The mini-project requires you to build an end-to-end deep learning pipeline **on your own**. This includes loading and preprocessing data, designing a model architecture, training the model, evaluating its performance, and analyzing the results. Instead of submitting a traditional paper, you will submit a Colab or Jupyter notebook that combines written explanations of your project with executable source code and the corresponding results. The structure of your notebook should resemble that of the homework assignments, and all cells in the notebook must be run to receive credit. The written component of your notebook should be around 3-4 pages in length, but there are no strict requirements on its length. We recommend that you structure your code similar to the homework assignments, where the majority of your implementation is in separate .py files and the notebook primarily consists of driver code that calls functions defined in the .py files.

Your notebook should comprise the following sections:

- **Introduction:** In this section, you should introduce the problem you are addressing, explain why it is significant or beneficial, and provide a summary of any relevant prior research or work that you are building upon.
- **Dataset:** In this section, you should describe the dataset or datasets you are using, present examples from them, and provide relevant information such as how you obtained the data, the labeling process, and the rationale behind your data collection decisions. If you created your own dataset, explain how you did it and why.
- **Method:** Here, you should describe the method you are using to solve the problem. You can also include parts of your model implementation, loss function, and other components, along with sanity checks to verify that they are correctly implemented. This section should be structured similarly to the homework assignments.
- **Experiments:** In this section, you should explain the experiments you performed, along with the key results we requested for each project. You can interleave explanations of the experiments, code for running them, and figures generated as a result of these experiments.
- **Conclusion/Future Work:** Here, you should provide a summary of your project, including what you learned, the limitations or failure cases of your work, and any ideas for future work or improvements. If you had more time or resources, explain how you would continue or expand upon the work you have already done.

Your submission should consist of a compressed .zip file, which includes your final notebook as well as all the code you wrote for the project. Please note that you should not include your training dataset in the compressed file, as it would make the file size too large to handle.

Open Source Code

Although there are numerous open source implementations available for the problems mentioned in the projects below, you should refrain from using them in your implementation. You may refer to existing open-source implementations only for specific details if you encounter confusion, but copying or importing code directly from existing implementations is strictly prohibited. Your implementation should solely utilize standard scientific computing libraries in Python, such as NumPy, SciPy, and Matplotlib, in addition to PyTorch, Torchvision, and libraries specified in each project's descriptions. If you have any questions regarding the usage of existing libraries, please ask them on our Moodle discussion forum.

Proposing your own project

We have prepared descriptions of several suggested projects for you. These are meant to be general guidelines; you don't have to implement every single idea that we mention in these descriptions.

If you have a project idea that you would like to pursue, please submit a project proposal of approximately 1-2 pages in length that addresses the following questions:

- What is the goal or task you want to achieve with your project?
- What are the main pieces of prior work you will build upon or reimplement? This may be one or multiple research papers.
- What data will you use for training and evaluation? If you are creating your own dataset, please describe the data collection process in detail.
- What computational resources will you use? Will you be using Colab or other computing resources?
- How will you evaluate your results? What datasets or evaluation metrics will you use? What criteria will you use to determine whether your model is working as expected?
- What are the key deliverables you plan to achieve? This should be a specific result, metric, or plot that you see as the primary outcome of your project. You will be evaluated based on whether you achieve this goal.

The list of offer projects:

1) Image classification

The goal here is to understand the importance of transfer learning from pre-trained models in real-world deployments of computer vision. You should build an image classification system using this technique, which involves collecting an image classification dataset, dividing it into splits, and training classification models by transfer learning from networks pretrained on ImageNet. Below you will find some papers to read to gain background on the problem and attribution methods. The first task is to collect an image classification dataset for a classification task of the reader's choice. you must define categories, collect and label images and split the dataset into train, validation, and test. Then you should train several models using torchvision, which provides various commonly used models, and compare multiple architectures. we recommend using ResNet18, ResNet50, or a small RegNet model (e.g. RegNetY-400MF).

Finally, You can consider three different ways of training a classifier:

- **Train from scratch:** Randomly initialize the model and train from scratch on your training set.
- **Feature extraction:** Initialize the model using weights pretrained on ImageNet; extract features from the model and train a linear model on top of extracted features. This can be implemented in several ways: you can use PyTorch to add a new layer onto the model, freeze all but the last layer, and train in PyTorch; you could also extract features from the model and save them in some format, then train a linear classifier with another package such as scikit-learn.
- **Fine-Tuning:** Initialize the model using weights pretrained on ImageNet; fine-tune the entire model on your training set. This should perform at least as well as feature extraction.

You may need to experiment with different optimizers, learning rates, L2 regularization strengths, and data augmentation strategies. **You should train at least 2 different models, and report their accuracy on both the train and val sets.**

2) Alzheimer's Detection

OASIS Alzheimer's Detection

Large-scale brain MRI dataset for deep neural network analysis

The dataset used is the OASIS MRI dataset, which consists of 80,000 brain MRI images. The images have been divided into four classes based on Alzheimer's progression. The dataset aims to provide a valuable resource for analyzing and detecting early signs of Alzheimer's disease.

To make the dataset accessible, the original .img and .hdr files were converted into Nifti format (.nii) using FSL (FMRIB Software Library). The converted MRI images of 461 patients have been uploaded to a GitHub repository, which can be accessed in multiple parts.

For the neural network training, 2D images were used as input. The brain images were sliced along the z-axis into 256 pieces, and slices ranging from 100 to 160 were selected from each patient. This approach resulted in a comprehensive dataset for analysis.

Patient classification was performed based on the provided metadata and Clinical Dementia Rating (CDR) values, resulting in four classes: demented, very mild demented, mild demented, and non-demented. These classes enable the detection and study of different stages of Alzheimer's disease progression.

During the dataset preparation, the .nii MRI scans were converted to .jpg files. Although this conversion presented some challenges, the files were successfully processed using appropriate tools. The resulting dataset size is 1.3 GB.

With this comprehensive dataset, the project aims to explore various neural network models and achieve optimal results in Alzheimer's disease detection and analysis.

Goals:

1. Train simple **SVM classification model** – Report the accuracy of the model and the Confusion matrix – try to use regularization (L2 or L1) and see if helps.
2. Construct **simple 2-layers convolutional network** – and report loss and accuracy..
3. Construct deep network (>2 layers) , with max-pooling, batch normalization, and tune it for best performance

OR

Use Pre-trained network, and apply transfer learning. (see LoRA)

3) Facial Expression

The data consists of 48x48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centered and occupies about the same amount of space in each image.

The task is to categorize each face based on the emotion shown in the facial expression into one of seven categories (0=Angry, 1=Disgust, 2=Fear, 3=Happy, 4=Sad, 5=Surprise, 6=Neutral). The training set consists of 28,709 examples and the public test set consists of 3,589 examples.

Goals:

1. Train simple **SVM classification model** – Report the accuracy of the model and the Confusion matrix – try to use regularization (L2 or L1) and see if helps.
2. Construct **simple 2-layers convolutional network** – and report loss and accuracy..
3. Construct your best deep network (>2 layers), and tune it for best performance.

OR

Use Pre-trained network, and apply transfer learning (training only one-layer) at the top . (see LoRA)

4) X-Ray Images (Pneumonia)

The dataset is organized into 3 folders (train, test, val) and contains subfolders for each image category (Pneumonia/Normal). There are 5,863 X-Ray images (JPEG) and 2 categories (Pneumonia/Normal).

Chest X-ray images (anterior-posterior) were selected from retrospective cohorts of pediatric patients of one to five years old from Guangzhou Women and Children's Medical Center, Guangzhou. All chest X-ray imaging was performed as part of patients' routine clinical care.

For the analysis of chest x-ray images, all chest radiographs were initially screened for quality control by removing all low quality or unreadable scans. The diagnoses for the images were then graded by two expert physicians before being cleared for training the AI system. In order to account for any grading errors, the evaluation set was also checked by a third expert.



The normal chest X-ray (left panel) depicts clear lungs without any areas of abnormal opacification in the image. Bacterial pneumonia (middle) typically exhibits a focal lobar consolidation, in this case in the right upper lobe (white arrows), whereas viral pneumonia (right) manifests with a more diffuse “interstitial” pattern in both lungs.

[http://www.cell.com/cell/fulltext/S0092-8674\(18\)30154-5](http://www.cell.com/cell/fulltext/S0092-8674(18)30154-5)

Goals:

1. Train simple **SVM classification model** – Report the accuracy of the model and the Confusion matrix – try to use regularization (L2 or L1) and see if helps.
2. Construct **simple 2-layers convolutional network** – and report loss and accuracy..
3. Construct deep network (>2 layers), with max-pooling, batch normalization, and tune it for best performance.

Or

Use Pre-trained network, and apply transfer learning ((see LoRA)

Note: Special attention will be given to the uniqueness of the work!

It's prohibited to copy or to use any other codes!